

Sincronización y comunicación por paso de mensajes

- Introducción
- Consideraciones generales
 - Modelos de sincronización
 - Referencia al emisor/receptor
- Paso de mensajes local
 - Descripción a nivel de lenguaje: ADA
 - Mensajes en POSIX 1003.1a: “Pipes” y FIFOs
 - Mensajes en POSIX 1003.1b: Colas de mensajes

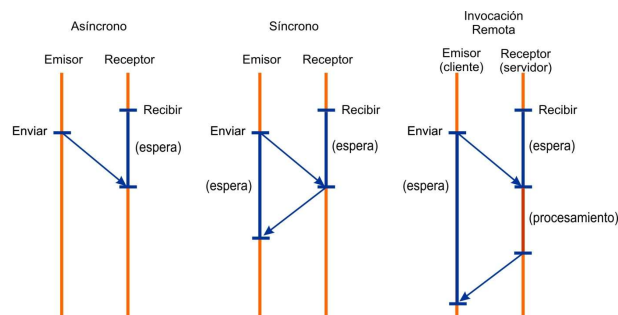
25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

1

Modelos de sincronización

- Modelos de sincronización (según funcionamiento de la operación de envío (**send**))
 - **Asíncrono**: El emisor no espera a que los datos sean recibidos. Si no queremos aceptar que se pierdan mensajes es necesaria una **cola de espera** (es lo que sucede en POSIX)
 - **Síncrono**: El emisor espera a que se produzca la recepción.
 - **Invocación remota**: Protocolo de llamada-respuesta. (ADA, “rendezvous”)



25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

3

Modelos de sincronización y referencia al emisor/receptor

- **Modelos de sincronización**
 - El modelo asíncrono puede utilizarse para emular los otros dos mediante un protocolo de mensajes
 - Inconvenientes del modelo asíncrono:
 - Necesidad de una zona de almacenamiento intermedio (cola) que puede llenarse
 - Dificultad para verificar la corrección del sistema
 - Menor eficiencia si se utiliza para emular otros modelos
- **Referencia al emisor/receptor**
 - Según el tipo de asociación entre las dos partes implicadas:
 - **Explícita:** En la orden de envío/recepción se especifica un identificador de la otra parte: "**send** <proceso destinatario>".
 - **Indirecta:** En la orden de envío/recepción se especifica el identificador de una estructura intermedia, como por ejemplo una cola de recepción: "**send** <cola>".
 - Situación en los dos procesos (emisor y receptor):
 - **Simétrica:** En ambos lados la referencia es del mismo tipo (explícita o indirecta).
 - **Asimétrica:** No se realiza la referencia del mismo modo

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

4

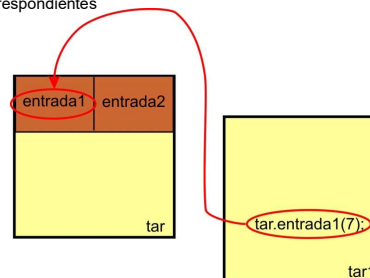
Descripción a nivel de lenguaje: ADA

- **"Entry":** Concepto de ADA que implementa el paso de mensajes

```
-- Tarea tar; posee varias "entry"
task tar is --Esta es la especificación "externa" de la tarea
  entry entrada1(l: in Integer);
  entry entrada2 .....
  ....
end tar;

task body tar is
  <sentencias>
  accept entrada1(l: in integer) do --Aquí se acepta el mensaje y se realizan las
    <tratamiento del mensaje>      -- operaciones orrespondientes
  end entrada1;
  <sigue el cuerpo de la tarea>
end tar;

-- La tarea tar1 utiliza una entry de la tarea tar
task body tar1 is
  <sentencias>
  tar.entrada1(7);
  <más sentencias>
end tar1;
```



25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

5

Descripción a nivel de lenguaje: ADA

- Sentencia **select**: espera simultánea de varias “**entry**” (se ejecuta el código de la primera que reciba datos)

```

<sentencias>
select
  accept entrada1(l: in integer) do
    <sentencias>
  end entrada1;
or
  accept entrada2(.....) do
    <sentencias>
  end entrada2;
  <resto de posibilidades>
or
  (...)
end select;
<más sentencias>

```

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

6

Paso de mensajes local en POSIX

- Varias opciones:
 - “Pipes” (POSIX 1003.1a)
 - FIFOs (POSIX 1003.1a)
 - Colas de mensajes (POSIX 1003.1b)
 - “Sockets” (también pueden utilizarse localmente)
- “Pipes”:
 - Cola circular (FIFO)
 - Acceso similar a un fichero utilizando las llamadas de bajo nivel (**read**, **write**), aunque **con bloqueo** cuando no se puede realizar la operación (cola vacía o cola llena)
 - Se construyen en la fase de inicialización, entre **fork** y **exec**
 - Inconvenientes:
 - Mecanismo de creación poco intuitivo
 - Son unidireccionales
 - En la práctica es necesario que los procesos implicados tengan un “antepasado” común
 - Fácil de utilizar desde el intérprete de comandos para crear un “pipeline” de comandos, conectando salidas y entradas “standard”:

ls | grep “.c” | wc

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

7

Paso de mensajes local en POSIX

- “FIFOs”:
 - Concepto similar a los “pipes”, pero más flexible y más fácil de utilizar
 - La “FIFO” es un objeto que existe en el sistema de archivos y **tiene un nombre**
 - Cualquier proceso con permisos adecuados puede crearla o abrirla como un fichero normal para enviar (**write**) o recibir (**read**) datos
 - Son objetos que persisten hasta ser explícitamente destruidos, aunque desaparezcan los procesos que los utilizaron
 - No existe el concepto de mensaje: Emisor y receptor pueden enviar y recibir números de octetos distintos en sus operaciones
- Inconvenientes comunes de “FIFOs” y “pipes”:
 - No existe el concepto de prioridad de mensaje
 - No se puede conocer o modificar fácilmente el espacio disponible en la cola
 - No se puede conocer el estado de la cola (número de datos encolados)
 - No se puede definir una estructura de mensaje, ni siquiera un tamaño
 - Número de colas limitado por el máximo número de archivos que pueden abrirse simultáneamente
 - Al estar asociadas al sistema de archivos, la implementación puede ser ineficiente

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

8

Paso de mensajes local en POSIX

- Colas de mensajes POSIX 1003.1b:
 - Medio de comunicación bastante más adecuado que las “FIFOs”
 - Creación y manejo muy similar al de los ficheros, pero con un conjunto de llamadas separado, y espacio de nombres propio (no tienen por qué “verse” en el sistema de archivos)
 - La llegada de mensajes puede ser notificada mediante señales
- Superan varios de los inconvenientes de las “FIFOs”:
 - Cada mensaje tiene una **prioridad** asociada y se reciben antes los de prioridad más alta, si están encolados
 - Al crear la cola se define explícitamente un **tamaño máximo de mensaje y una capacidad** (máximo número de mensajes encolados)
 - Puede conocerse en cualquier momento el **número de mensajes encolados**
 - Aunque no existe estructura de mensaje, **si se conoce su tamaño**. Un mensaje se envía y recibe obligatoriamente con una sola llamada
 - La implementación de las colas puede ser independiente de la de los archivos, y por tanto **más eficiente y con menos limitaciones**

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

9

POSIX: Permisos y flags

- Llamada **open**:

```
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
```

```
int open(const char *nombre, int flag, mode_t modo);
```

O_RDONLY
O_WRONLY
O_RDWR
O_CREAT
O_EXCL
O_APPEND
O_TRUNC
O_NONBLOCK

S_I + (R,W,X) + (USR, GRP; OTH)
S_IRWXU
S_IRWXG
S_IRWXO

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

10

POSIX: Relación entre diferentes llamadas

Funciones	Archivos ANSI-C	Archivos bajo nivel	Mensajes	Semáforos
Abrir/Crear	fopen	open	mq_open	sem_open
Cerrar	fclose	close	mq_close	sem_close
Acceder	fread/fwrite	read/write	mq_receive/ mq_send	sem_post/ sem_wait
Destruir	remove	unlink	mq_unlink	sem_unlink

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

11

Colas de mensajes POSIX 1003.1b

- Creación y/o apertura:


```
mqd_t mq_open(char *mq_name, int oflag, mode_t modo, struct mq_attr *atr);
```

 - Como **open**, pero añade estructura de atributos
- Operaciones básicas:
 - Envío:


```
int mq_send(mqd_t cola, char *datos, size_t longitud, int prioridad);
```
 - Recepción:


```
int mq_receive(mqd_t cola, char *datos, size_t longitud, int *prioridad);
```
 - Envío y recepción similares a **write** y **read**, pero con prioridad
 - Ambas **son bloqueantes por defecto** cuando no hay datos (**mq_receive**) o no hay espacio (**mq_send**)
- Muestreo del estado:


```
int mq_getattr(mqd_t cola, struct mq_attr *atr);
```
- Cierre y destrucción:


```
int mq_close(mqd_t cola);
int mq_unlink(char *nombre);
```

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

12

Colas de mensajes POSIX 1003.1b

```
mqd_t mq_open(char *name, int oflag, mode_t modo, struct mq_attr *atr);
```

- Nombre:
 - Carácter "/" sólo al principio: "/cola1"
 - No tiene por qué existir en el sistema de archivos
- Flags:
 - O_RDONLY, O_WRONLY, O_RDWR: Tipo de acceso permitido. Han de ser compatibles con los permisos del usuario
 - O_CREAT: Se permite crear cola
 - O_EXCL: Se prohíbe no crear la cola (si existe, error)
 - O_NONBLOCK: Envío y recepción no bloqueantes
- Modo:
 - Sólo útil si se crea la cola, si no es ignorado
 - Contiene los permisos de acceso para usuario y grupo
- Estructura de atributos:
 - Sólo útil si se crea la cola, si no es ignorado
 - Permite fijar las características de la cola:
 - Tamaño máximo de mensaje en octetos ("bytes")
 - Número máximo de mensajes encolados (capacidad de la cola)
 - Los otros campos se ignoran en **mq_open**

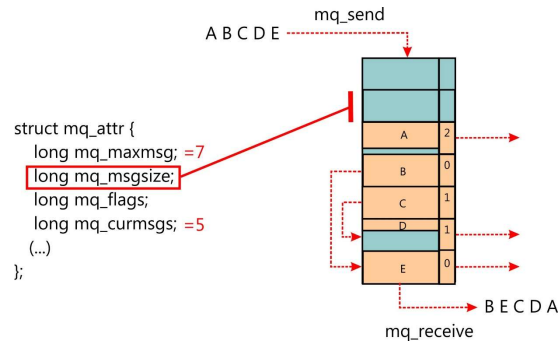
25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

13

Colas de mensajes POSIX 1003.1b

- **Estructura de atributos:**
 - Permite fijar la capacidad (**mq_maxmsg**) y el tamaño del mensaje (**mq_msgsize**)
 - Permite conocer el número de mensajes encolados (**mq_curmsgs**) usando **mq_getattr**
- **Prioridad:**
 - Se recibe con el mensaje
 - Funcionamiento similar a varias colas, una por prioridad



25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

14

Colas de mensajes POSIX 1003.1b

- **Envío de datos:**

```
int mq_send(mqd_t cola, char *datos, size_t longitud, int pri);
```

 - Envío de **longitud** octetos existentes dónde apunta **datos** con prioridad **pri**
 - El valor de **longitud** ha de ser menor o igual que **mq_msgsize**
 - Devuelve 0 o -1 (error)
 - Bloqueante salvo activación de O_NONBLOCK (si la cola está llena, **espera a que no lo esté**)
- **Recepción de datos:**

```
int mq_receive(mqd_t cola, char *datos, size_t longitud, int *ppri);
```

 - Recepción de mensaje sobre la zona de memoria a la que apunta **datos**, con tamaño **longitud**.
 - La prioridad se recibe por medio de **ppri**, si no vale **NULL**
 - El valor de **longitud** ha de ser igual o mayor que **mq_msgsize**
 - Devuelve el **número de octetos recibido** o error (-1)
 - Bloqueante salvo activación de O_NONBLOCK (si la cola está vacía, **espera a que no lo esté**)

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

15

Colas de mensajes POSIX 1003.1b

- Muestreo del estado:

```
int mq_getattr(mqd_t cola, struct mq_attr *atr);
```

- Permite conocer el estado de una cola previamente abierta:
 - Tamaño máximo de mensaje y capacidad de la cola (estáticos)
 - Número de mensajes almacenados en la cola (dinámico)

- Cierre y destrucción:

```
int mq_close(mqd_t cola);
```

```
int mq_unlink(char *nombre);
```

- Cerrar la cola (**mq_close**) no la destruye
- La destrucción (**mq_unlink**) es “diferida”:
 - La cola pierde su nombre; nadie más puede abrirla
 - Desaparece solo **cuando la cierra el último** que la tiene abierta

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

16

Colas de mensajes POSIX 1003.1b

- Notificación asíncrona

```
int mq_notify(mqd_t cola, struct sigevent *espec);
```

- Permite hacer que una cola **vacía** genere un evento (señal) cuando llegan datos, teniendo en cuenta que
 - No puede haber ningún **mq_receive** esperando
 - Sólo sirve para una señal; después hay que repetir **mq_notify**
 - Sólo un proceso puede programarla a la vez
 - Nunca se generará la señal cuando la cola no estaba vacía antes de llegar los datos
- Utilidad: Permite manejar varias colas a la vez
 - Cada vez que salta la señal reaccionamos para leer los datos
 - La señal puede ser compartida, o podemos esperar varias señales a la vez
- Si pueden utilizarse varios hilos su uso puede sustituirse utilizando un hilo dedicado a cada cola
- Ni **mq_notify** ni las otras llamadas asociadas a las colas tienen por qué ser seguras en un manejador

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

17

Mensajes de POSIX 1003.1b: Notificación Asíncrona

```
#define _POSIX_C_SOURCE 199309L
#include <unistd.h>
#include <mqueue.h>
#include <signal.h>

void manejador(int sig, siginfo_t *info, void *nada) { }
void main(void) {
    struct sigevent espec; struct sigaction accion;
    siginfo_t info; mqd_t cola; mqd_t *pcola; sigset_t sig;

    /* Abrir la cola */
    cola = mq_open(...);

    /* Configurar señal como de tiempo real */
    accion.sa_sigaction = manejador;
    accion.sa_flags = SA_SIGINFO;
    sigemptyset(&accion.sa_mask);
    sigaction(SIGRTMIN, &accion, NULL);

    /* Enmascarar señal */
    sigemptyset(&sig); sigaddset(&sig, SIGRTMIN);
    sigprocmask(SIG_BLOCK, &sig, NULL);

    /* Conectar la cola de mensajes con la señal
       si se desean mas notificaciones tiene que hacerse lo
       mismo al tratar la señal */
    espec.sigev_notify = SIGEV_SIGNAL;
    espec.sigev_signo = SIGRTMIN;
    espec.sigev_value.sival_ptr = (void *)&cola; /* puntero a
       cola */
    mq_notify(col, &espec);

    (...)

    sigwaitinfo(&sig, &info);
    pcola = (mqd_t *)info.si_value.sival_ptr;
    <preparar sigevent espec>
    mq_notify(*pcola, &espec); /* Puede fallar si otro
       proceso lo intenta */
    <leer todos los mensajes, hasta
       dejar vacía la cola>
}
```

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

18

Colas de mensajes: Puntos a recordar

- Abrir y crear la cola:
 - Normalmente interesa crear la cola, no usar una existente
 - Diferentes maneras de hacerlo:
 - Destruir la cola antes de intentar crearla
 - Intentar crearla con flag EXCL y destruirla si ya existe
 - Un proceso/hilo crea y abre la cola (O_CREAT) el resto sólo la abre
- Recibir datos (**mq_receive**):
 - Nuestro “buffer” debe ser igual o mayor que el tamaño máximo de mensaje
 - La lectura es **bloqueante** (salvo con O_NONBLOCK)
 - Si la cola está vacía **mq_receive** espera a que deje de estarlo, salvo si lo interrumpe un manejador
 - Los mensajes pueden tener un tamaño menor que el máximo, y podemos conocerlo (es lo que devuelve **mq_receive**)
 - Los mensajes más prioritarios se anticipan a los menos prioritarios, pero eso sólo sucede si se acumulan en la cola

25/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

19

Colas de mensajes: Puntos a recordar

- **Enviar datos (mq_send):**
 - Nuestro “buffer” debe ser menor o igual que el tamaño máximo de mensaje
 - El envío es **bloqueante** (salvo con O_NONBLOCK)
 - Si la cola está llena **mq_send** espera a que deje de estarlo, salvo si lo interrumpe un manejador
 - Podemos enviar mensajes con tamaño menor que el máximo
- **Destruir/cerrar colas:**
 - **mq_close** no destruye la cola; sólo la cierra
 - **mq_unlink** no destruye inmediatamente la cola; los que la tienen abierta pueden seguir usándola
- **mq_getattr:**
 - Permite saber la ocupación actual de la cola – si está llena o vacía, por ejemplo
 - Permite saber la capacidad y el tamaño máximo del mensaje (útil si no la hemos creado)